

Somador de Ponto Flutuante Simplificado em FPGA

Dossiê Completo de Projeto, Operação e Apresentação

Formato de 13 bits · Entrada sequencial dos 26 bits
Placa Terasic DE10-Lite · Intel MAX 10 – 10M50DAF484C7G

MCTA024 - Sistemas Digitais

Universidade Federal do ABC

Profa. Victoria Alejandra Herrera

Base teórica: Pong P. Chu, *FPGA Prototyping by VHDL Examples*, seção 3.7.4

Sumário

1	O que é o projeto e o formato de 13 bits
2	Como colocar o programa em uso — passo a passo no Quartus
3	O painel: exatamente o que cada chave e cada botão faz
4	Receita A — converter um número decimal em posições de chave
5	Receita B — converter o display hexadecimal em decimal
6	Exemplo completo resolvido do início ao fim
7	Como a soma é feita — os quatro estágios em profundidade
8	Como a normalização é feita — nas duas pontas do circuito
9	Os limites do formato — até quanto o somador vai
10	Perguntas prováveis da professora, com respostas prontas
11	O que foi mudado em relação ao projeto original do livro
12	Validação: como sabemos que funciona
13	Problemas comuns e o que fazer
14	Arquivos do projeto
15	Resumo de bolso para a apresentação

1. O que é o projeto e o formato de 13 bits

O projeto é um **circuito digital que soma dois números em ponto flutuante**, implementado em VHDL e gravado numa FPGA. O formato usado é uma versão simplificada de 13 bits, definida no enunciado da disciplina.

1.1 A estrutura dos 13 bits

| s | e3 e2 e1 e0 | f7 f6 f5 f4 f3 f2 f1 f0 |
1 4 8 = 13 bits
valor = $(-1)^s \times 0.f \times 2^e$

Campo	Bits	Significado
s	1	Sinal. 0 = positivo, 1 = negativo.
e	4	Expoente, sem sinal . Vai de 0 a 15.
f	8	Fração, sem sinal . O valor 0.f equivale a f/256.

Como o sinal está separado da magnitude, este formato é uma variação de **sinal-magnitude** — e não complemento de dois. Essa escolha tem consequências diretas no Estágio 3 do somador (seção 7.3).

1.2 A regra de normalização

Um número neste formato só é válido se uma destas duas condições for verdadeira:

- O bit mais significativo da fração (f7) vale 1; **ou**
- O número é exatamente o **zero canônico**: e = 0000 e f = 00000000.

Por que essa regra existe? Para garantir **representação única**. Sem ela, o mesmo valor teria várias formas possíveis:

$0.11000000 \times 2^{0110} = 192/256 \times 64 = 48$
 $0.01100000 \times 2^{0111} = 96/256 \times 128 = 48$ <-- mesmo valor, outra forma
 $0.00110000 \times 2^{1000} = 48/256 \times 256 = 48$ <-- mesmo valor, outra forma
so a PRIMEIRA é normalizada (f7 = 1)

A regra também tem uma consequência prática importante: como f7 é sempre 1, a fração vale sempre entre **128 e 255** quando vista como número inteiro. Isso dá uma intuição muito útil para operar a placa:

Atalho mental: pense na fração como um **inteiro de 128 a 255**. Então $\text{valor} = \text{fração} \times 2^{(\text{expoente} - 8)}$. Quando o expoente é exatamente 8, o valor é a fração. Toda a Receita A da seção 4 nasce dessa relação.

1.3 Alcance do formato

Grandeza	Representação	Valor
Maior magnitude	$0.11111111 \times 2^{1111}$	$255 \times 128 = \mathbf{32.640}$
Menor magnitude não nula	$0.10000000 \times 2^{0000}$	$128 \times 2^{-8} = \mathbf{0,5}$
Zero	$0.00000000 \times 2^{0000}$	0

A faixa total é de aproximadamente 2^{16} , que é muito mais do que 13 bits em inteiro com sinal conseguiriam cobrir — e essa é justamente a vantagem do ponto flutuante. O preço é a **precisão variável**, detalhada na seção 9.

2. Como colocar o programa em uso

Este procedimento é feito **uma vez**. Depois é só operar as chaves. Tempo estimado: 10 a 15 minutos, sendo a compilação a parte mais demorada.

2.1 Antes de começar

Você precisa de	Observação
Quartus Prime Lite	A edição Lite é gratuita e suporta a família MAX 10. Não precisa de licença.
Placa DE10-Lite	Com o cabo USB tipo B (o conector quadrado, à esquerda da placa).
somador_pf_de10lite_seq.vhd	O programa. Autocontido — é o único arquivo de código necessário.
somador_pf_de10lite_seq.qsf	As atribuições de pino.
somador_pf_de10lite_seq.sdc	A restrição de tempo. Obrigatória, porque esta versão tem clock.

2.2 Passo 1 — Criar o projeto

Abra o Quartus e vá em `File` → `New Project Wizard`. Preencha:

Campo do assistente	O que colocar
Working directory	Uma pasta vazia sua. Evite caminhos com acento ou espaço.
Project name	somador_pf_de10lite_seq
Top-level entity	somador_pf_de10lite_seq — tem que ser exatamente igual ao nome da entidade no VHDL, senão não compila.
Project type	Empty project
Add files	Adicione apenas somador_pf_de10lite_seq.vhd.
Family	MAX 10 (DA/DF/DC/DE/DT/SA/SC/SE/ST)
Device	10M50DAF484C7G — use o filtro de nome para achar.
EDA Tool Settings	Pule. (Se for usar o Questa, marque em <i>Simulation</i> .)

Clique em `Finish`.

2.3 Passo 2 — Importar as atribuições de pino

`Assignments` → `Import Assignments...` → selecione o arquivo `somador_pf_de10lite_seq.qsf` → `OK`.

Confira antes de compilar. Abra `Assignments` → `Pin Planner` e verifique se `MAX10_CLK1_50` está no pino **PIN_P11**. Se o clock estiver no pino errado, o circuito simplesmente não funciona e não há mensagem de erro — só um projeto que compila e nada acontece na placa.

2.4 Passo 3 — Adicionar a restrição de tempo

`Project` → `Add/Remove Files in Project...` → adicione `somador_pf_de10lite_seq.sdc` → `OK`.

Esse arquivo diz ao Quartus que o clock é de 50 MHz (período de 20 ns). Sem ele, o analisador de tempo reclama de "clock não restringido" e não valida o projeto.

2.5 Passo 4 — Compilar

`Processing` → `Start Compilation` (atalho `Ctrl+L`). Leva de 2 a 5 minutos.

Ao terminar, olhe a aba `Messages`. Você precisa de **0 erros**. *Warnings* são normais e esperados neste projeto. Se der erro:

Mensagem	Causa
"Top-level design entity ... is undefined"	O nome do top-level não bate com o da entidade no VHDL.
"Can't find ... pin assignment"	O <code>.qsf</code> não foi importado.
"Error ... VHDL syntax error"	O arquivo foi copiado incompleto. Baixe de novo.

2.6 Passo 5 — Gravar na placa

- Conecte a DE10-Lite ao computador pelo **cabo USB**.
- Confirme que o LED `POWER` da placa está aceso.

3. `Tools` → `Programmer`.
4. Clique em `Hardware Setup...`. Na lista, selecione **USB-Blaster [USB-0]** e feche. (Se a lista estiver vazia, veja a seção 13.)
5. Confirme que `Mode` está em **JTAG**.
6. Se a lista de dispositivos estiver vazia, clique `Auto Detect` e escolha `10M50DA`.
7. `Add File...` → navegue até `output_files/` e selecione `somador_pf_de10lite_seq.sof`.
8. Marque a caixa **Program/Configure** na linha do arquivo.
9. Clique em **Start**.
10. A barra de progresso deve chegar a **100% (Successful)**.

Como saber que deu certo. Assim que a gravação termina, os displays acendem mostrando:

HEX5	HEX4	HEX3	HEX2	HEX1	HEX0
1	-	-	8	0.	9

Isso acontece porque os registradores nascem com +128 nos dois operandos, e $128 + 128 = 256 = 0.10000000 \times 2^9$, que em hexadecimal é `80` com expoente `9`. Se você vê `80.9`, está funcionando.

ATENÇÃO A gravação `.sof` é **volátil**: ela se perde quando a placa é desligada. Para a apresentação isso não é problema — basta regravar. Para gravação permanente, use `File` → `Convert Programming Files` e gere um `.pof`.

3. O painel: o que cada chave e cada botão faz

A placa tem **10 chaves deslizantes** (SW9 a SW0) e **2 botões** (KEY1 e KEY0). Este projeto usa todos os 12.

O conceito central. Nesta versão as chaves não são fios permanentes ligados aos bits do somador — elas funcionam como um **teclado**. Você posiciona as chaves, aperta KEY0, e o valor é *copiado* para dentro de um registrador. Depois pode mexer nas chaves livremente: o valor guardado não muda. Repetindo isso 4 vezes, você entra os 26 bits completos dos dois operandos.

3.1 Os dois seletores: SW9 e SW8

Essas duas chaves **não carregam dados**. Elas decidem *onde* a próxima carga vai cair.

Chave	Função
SW9	Qual número: para baixo = número 1 · para cima = número 2
SW8	Qual campo: para baixo = sinal + expoente · para cima = fração

Combinando as duas, obtêm-se as 4 fases de digitação. O display HEX5 mostra em qual fase você está, para não haver dúvida:

SW9	SW8	HEX5 mostra	O que o KEY0 vai carregar
baixo	baixo	1	sinal e expoente do número 1
baixo	cima	2	fração do número 1
cima	baixo	3	sinal e expoente do número 2
cima	cima	4	fração do número 2

A ordem das fases não importa. Você pode carregar na sequência que quiser e recarregar um campo isolado quantas vezes quiser — útil para, por exemplo, mudar só o expoente de um operando e ver o efeito na hora.

3.2 As oito chaves de dados: SW7 a SW0

São **reaproveitadas** nas quatro fases. O significado depende da posição do SW8:

Quando SW8 está para BAIXO (fase de sinal + expoente)

SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
ignorada	ignorada	ignorada	SINAL cima = negativo	8	4	2	1
tanto faz a posição			SW3..SW0 formam o expoente de 4 bits (0 a 15)				

Quando SW8 está para CIMA (fase de fração)

SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
128	64	32	16	8	4	2	1

SW7 deve estar sempre para cima na fase de fração. É o bit f7, e ele precisa valer 1 para o número estar normalizado. Se você esquecer, o circuito *corrige automaticamente* (seção 8.1) e acende o LEDR8 para avisar — mas o número que entra na conta não é o que você imaginou.

3.3 Os dois botões

Botão	O que faz
KEY0	CARREGA. Copia as chaves de dados para o campo selecionado por SW9/SW8. Um clique = uma carga. Segurar o botão apertado não recarrega (há detector de borda). Há também anti-ripique de 10 ms, então não existe carga dupla acidental.
KEY1	CONFERÊNCIA, enquanto estiver apertado. Os displays param de mostrar o resultado e passam a mostrar o operando <i>guardado</i> escolhido pelo SW9. Aparece a letra C no HEX4 avisando. Ao soltar, volta ao resultado. Use isso sempre antes de anunciar um resultado: em dois segundos você confirma que digitou os dois operandos certos.

Não existe botão de *reset*. Ele não é necessário: qualquer campo pode ser sobrescrito a qualquer momento, e os registradores ligam com um valor válido de fábrica.

3.4 Os seis displays

HEX5	HEX4	HEX3	HEX2	HEX1	HEX0
fase atual (1 a 4)	letra C na conferência, senão apagado	sinal – ou apagado	fração bits 7 a 4	fração bits 3 a 0 + ponto decimal	expoente

Lê-se da esquerda para a direita como notação científica. Por exemplo, 4 – D 8 . 5 significa: *estou na fase 4, e o valor é $-0.D8_{hex} \times 2^5$* , que dá –27.

Sim, o display é hexadecimal. A fração aparece como dois dígitos hex (00 a FF) e o expoente como um dígito hex (0 a F). A seção 5 ensina a converter. O **ponto decimal aceso** no HEX1 é apenas o separador visual entre fração e expoente.

3.5 Os dez LEDs

LED	O que indica
LEDR9	O valor mostrado é negativo.
LEDR8	Aviso: uma das entradas foi corrigida pelo pré-normalizador. Você digitou um número fora do formato normalizado.
LEDR7 a LEDR0	A fração mostrada, em binário . É a sua leitura sem precisar de hexadecimal. Pesos: 128, 64, 32, 16, 8, 4, 2, 1.

Dois detalhes úteis: LEDR7 aceso é a **prova visual de que a saída está normalizada** (o MSB da fração é 1). E se todos os oito estiverem apagados, o resultado é zero.

O display atualiza a cada clique. Tudo depois dos registradores é combinacional, então o resultado muda imediatamente. Consequência prática: *durante* a digitação você vai ver resultados que parecem errados, porque metade dos campos ainda pertence ao número anterior. Isso é esperado. Só confie no display depois dos quatro cliques.

4. Receita A — decimal para posições de chave

Esta é a conta que você faz quando a professora diz "some tal número com tal número".

4.1 O método

Passo 1. Separe o sinal. Trabalhe só com o módulo (o número sem sinal).

Se for negativo, a chave **SW4** vai para cima na fase de sinal+expoente.

Passo 2. Ajuste o número até ele cair na faixa **128 a 255**:

- se for **maior ou igual a 256**, vá **dividindo por 2**;
- se for **menor que 128**, vá **multiplicando por 2**.

Passo 3. O número final é a **FRAÇÃO**.

Passo 4. Conte os passos que você deu:

```
EXPOENTE = 8 + (numero de divisoes)
EXPOENTE = 8 - (numero de multiplicacoes)
```

Passo 5. Converta os dois para binário:

- fração → 8 bits → chaves **SW7..SW0** na fase de fração;
- expoente → 4 bits → chaves **SW3..SW0** na fase de sinal+expoente.

Passo 6. Confira: o expoente tem que estar entre **0 e 15**. Se saiu fora, **o número não cabe no formato**.

Se em algum passo a divisão não der um número inteiro, **arredonde** — e anote, porque significa que aquele número não é exatamente representável.

4.2 Por que essa receita funciona

Não é mágica, é só álgebra. Queremos escrever o valor V como $(f/256) \times 2^e$ com $128 \leq f \leq 255$. Reorganizando:

$$V = (f / 256) \times 2^e = f \times 2^{(e-8)}$$

$$\text{Chamando } d = e - 8: \quad V = f \times 2^d \quad \text{ou seja} \quad f = V / 2^d$$

Dividir V por 2^d vezes e chegar na faixa 128..255 significa exatamente encontrar esse f . E daí: $e = d + 8$.

4.3 Exemplos resolvidos

Exemplo 1 — o número 1536 (positivo)

```
1536 / 2 = 768
768 / 2 = 384
384 / 2 = 192    <-- caiu entre 128 e 255. Foram 3 DIVISOES.
```

```
FRACAO = 192    = 1100 0000 (binario)
EXPOENTE = 8 + 3 = 11 = 1011 (binario)
```

Verificacao: $192 \times 2^{(11-8)} = 192 \times 8 = 1536$ EXATO

Exemplo 2 — o número 320 (positivo)

```
320 / 2 = 160    <-- caiu na faixa. 1 DIVISAO.
```

```
FRACAO = 160    = 1010 0000
EXPOENTE = 8 + 1 = 9 = 1001
```

Verificacao: $160 \times 2^{(9-8)} = 160 \times 2 = 320$ EXATO

Exemplo 3 — o número 27 (pequeno, precisa MULTIPLICAR)

```
27 x 2 = 54
54 x 2 = 108
108 x 2 = 216    <-- caiu na faixa. Foram 3 MULTIPLICACOES.
```

```
FRACAO = 216    = 1101 1000
EXPOENTE = 8 - 3 = 5 = 0101
```

Verificacao: $216 \times 2^{(5-8)} = 216 / 8 = 27$ EXATO

Exemplo 4 — o número 16.384 (grande)

```
16384 -> 8192 -> 4096 -> 2048 -> 1024 -> 512 -> 256 -> 128
              7 DIVISOES

FRACAO  = 128 = 1000 0000
EXPOENTE = 8 + 7 = 15 = 1111

Verificacao: 128 x 2^7 = 16384           EXATO
```

Exemplo 5 — o número 1500 (não cabe exato)

```
1500 / 2 = 750
 750 / 2 = 375
 375 / 2 = 187,5    <-- esta na faixa, mas NAO E INTEIRO

Arredondando para 188:
FRACAO  = 188 = 1011 1100
EXPOENTE = 11 = 1011

Verificacao: 188 x 8 = 1504           <-- erro de +4
```

Não é defeito do circuito: na faixa do expoente 11 os números existem apenas de 8 em 8 (...1496, 1504, 1512...). O 1500 simplesmente não está entre eles.

Exemplo 6 — o número 60.000 (impossível)

```
60000 -> 30000 -> 15000 -> 7500 -> 3750 -> 1875 -> 937,5 -> 468,75 -> 234,375
              8 DIVISOES

EXPOENTE = 8 + 8 = 16

16 NAO CABE em 4 bits (o maximo e 15).    >>> NUMERO IMPOSSIVEL <<<
```

4.4 Tabela de expoentes prontos

Exp	SW3 SW2 SW1 SW0	Exp	SW3 SW2 SW1 SW0
0	baixo baixo baixo baixo	8	cima baixo baixo baixo
1	baixo baixo baixo cima	9	cima baixo baixo cima
2	baixo baixo cima baixo	10	cima baixo cima baixo
3	baixo baixo cima cima	11	cima baixo cima cima
4	baixo cima baixo baixo	12	cima cima baixo baixo
5	baixo cima baixo cima	13	cima cima baixo cima
6	baixo cima cima baixo	14	cima cima cima baixo
7	baixo cima cima cima	15	cima cima cima cima

5. Receita B — hexadecimal para decimal

O display mostrou E8.B. E agora?

5.1 O método

Passo 1. Leia os dois dígitos hexadecimais da fração (HEX2 e HEX1) e converta para decimal:

$$\text{FRACAO} = (\text{dígito do HEX2}) \times 16 + (\text{dígito do HEX1})$$

Passo 2. Leia o expoente no HEX0 e converta para decimal.

Passo 3. Ajuste:

- se **expoente > 8**: **multiplique** a fração por 2, (expoente - 8) vezes;
- se **expoente < 8**: **divida** a fração por 2, (8 - expoente) vezes;
- se **expoente = 8**: o valor é a fração.

Passo 4. Se o HEX3 mostra um traço horizontal (-), o resultado é **negativo**.

5.2 Tabela de conversão hexadecimal

Hex	Binário	Decimal	Hex	Binário	Decimal
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	b	1011	11
4	0100	4	C	1100	12
5	0101	5	d	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

No display de 7 segmentos, o b e o d aparecem em minúsculo. É a única forma de não confundir b com 8 e d com 0.

5.3 Exemplos resolvidos

Display E8.B

```
FRACAO = E x 16 + 8 = 14 x 16 + 8 = 232
EXPOENTE = B = 11

11 - 8 = 3 -> multiplica por 2 tres vezes:
232 -> 464 -> 928 -> 1856

HEX3 apagado -> positivo

VALOR = +1856
```

Display -D8.5

```
FRACAO = D x 16 + 8 = 13 x 16 + 8 = 216
EXPOENTE = 5

8 - 5 = 3 -> divide por 2 tres vezes:
216 -> 108 -> 54 -> 27

HEX3 mostra "-" -> negativo

VALOR = -27
```

Display FE.F

```
FRACAO = F x 16 + E = 15 x 16 + 14 = 254
EXPOENTE = F = 15

15 - 8 = 7 -> multiplica por 2 sete vezes = x 128
254 x 128 = 32512

VALOR = +32512
```

Display 00.0

Fração 0 e expoente 0: é o **zero canônico**. O resultado é zero.

5.4 Alternativa sem hexadecimal

Se preferir não converter hexadecimal na hora da apresentação, olhe os **LEDs LEDR7 a LEDR0**: eles mostram a mesma fração em binário. Basta somar os pesos dos LEDs acesos.

LEDR7	LEDR6	LEDR5	LEDR4	LEDR3	LEDR2	LEDR1	LEDR0
128	64	32	16	8	4	2	1

No exemplo do E8 (232), acendem LEDR7, LEDR6, LEDR5 e LEDR3: $128 + 64 + 32 + 8 = 232$. Depois é só aplicar o Passo 3 da receita com o expoente.

6. Exemplo completo: a professora pede "1536 + 320"

6.1 Passo 1 — fazer as contas

Número 1		Número 2
Valor pedido	+1536	+320
Sinal	positivo → SW4 baixo	positivo → SW4 baixo
Divisões por 2	3 (1536→768→384→192)	1 (320→160)
Fração	192 = 1100 0000	160 = 1010 0000
Expoente	8+3 = 11 = 1011	8+1 = 9 = 1001

6.2 Passo 2 — os quatro cliques

C = chave para cima · **b** = chave para baixo · **x** = não importa

Clique 1 — sinal e expoente do número 1

SW9	SW8	SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
baixo	baixo	x	x	x	baixo	CIMA	baixo	CIMA	CIMA
nº 1	sinal+exp				sinal +	1	0	1	1

O HEX5 mostra **1.** → aperte **KEY0**

Clique 2 — fração do número 1 (192 = 11000000)

SW9	SW8	SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
baixo	CIMA	CIMA	CIMA	baixo	baixo	baixo	baixo	baixo	baixo
nº 1	fração	1	1	0	0	0	0	0	0

O HEX5 mostra **2.** → aperte **KEY0**

Clique 3 — sinal e expoente do número 2

SW9	SW8	SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
CIMA	baixo	x	x	x	baixo	CIMA	baixo	baixo	CIMA
nº 2	sinal+exp				sinal +	1	0	0	1

O HEX5 mostra **3.** → aperte **KEY0**

Clique 4 — fração do número 2 (160 = 10100000)

SW9	SW8	SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
CIMA	CIMA	CIMA	baixo	CIMA	baixo	baixo	baixo	baixo	baixo
nº 2	fração	1	0	1	0	0	0	0	0

O HEX5 mostra **4.** → aperte **KEY0**

6.3 Passo 3 — conferir antes de anunciar

- Coloque **SW9 baixo** e **segure KEY1**: o display mostra **C0.b**. Fração **C0** = 192, expoente **11** → $192 \times 8 = 1536$ ✓
- Coloque **SW9 cima** e **segure KEY1**: mostra **A0.9**. Fração **A0** = 160, expoente **9** → $160 \times 2 = 320$ ✓

Solte o KEY1 e volte ao resultado.

6.4 Passo 4 — ler o resultado

HEX5	HEX4	HEX3	HEX2	HEX1	HEX0
4	-	-	E	8.	b

Aplicando a Receita B:

FRACAO	= E8	= 14 x 16 + 8	= 232
EXPOENTE	= b	= 11	
11 - 8	= 3	-> 232 x 8	= 1856
HEX3 apagado	->	positivo	
RESULTADO	=	+1856	

Confere: $1536 + 320 = 1856$ ✓ exato, sem nenhuma perda de precisão.

Os LEDs confirmam: acendem LEDR7, LEDR6, LEDR5 e LEDR3, que somam $128+64+32+8 = 232$. O LEDR8 fica apagado, indicando que nenhuma das entradas precisou de correção.

6.5 Um segundo exemplo, com números grandes: 16.384 + 16.128

	Número 1	Número 2
Valor	+16.384	+16.128
Divisões por 2	7	6
Fração	$128 = 1000\ 0000 = 0x80$	$252 = 1111\ 1100 = 0xFC$
Expoente	$8+7 = 15 = 1111$	$8+6 = 14 = 1110$

Clique	SW9	SW8	SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
1 · sinal+exp nº1	b	b	×	×	×	b	C	C	C	C
2 · fração nº1	b	C	C	b	b	b	b	b	b	b
3 · sinal+exp nº2	C	b	×	×	×	b	C	C	C	b
4 · fração nº2	C	C	C	C	C	C	C	C	b	b

display: F E . F

FRACAO = FE = $15 \times 16 + 14 = 254$

EXPOENTE = F = 15

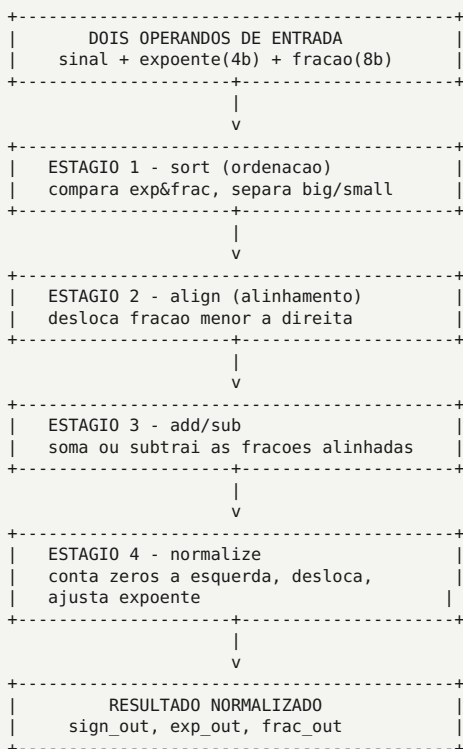
$15 - 8 = 7 \rightarrow 254 \times 128 = 32512$

RESULTADO = +32512 confere: $16384 + 16128 = 32512$ EXATO

Esse é um bom exemplo para a apresentação porque fica a apenas 128 do teto absoluto do formato.

7. Como a soma é feita — os quatro estágios

Esta é a seção para responder "como vocês fizeram a soma?". O somador é um circuito **100% combinacional**: não tem clock, não tem registrador, não tem máquina de estados. Os quatro "estágios" **não são estágios de pipeline** — são quatro blocos de lógica encadeados, e o resultado aparece na saída por propagação direta.



A lógica segue o mesmo procedimento que uma pessoa usa para somar à mão em notação científica. Os sufixos usados nos nomes dos sinais são a convenção do livro: **b** = *big* (maior), **s** = *small* (menor), **a** = *aligned* (alinhado), **n** = *normalized*.

7.1 Estágio 1 — sort (ordenação)

O que faz: descobre qual dos dois números tem a **maior magnitude** e o encaminha para os sinais `signb`, `expb`, `fracb`. O outro vai para `signs`, `exps`, `fracs`.

```
if unsigned(exp1 & frac1) > unsigned(exp2 & frac2) then
    signb <= sign1;  signs <= sign2;
    expb <= unsigned(exp1);  exps <= unsigned(exp2);
    fracb <= unsigned(frac1);  fracs <= unsigned(frac2);
else
    ... (trocado)
end if;
```

O truque esperto do livro, e uma boa resposta se a professora perguntar: a comparação é feita entre `exp` e `frac` **concatenados**, formando um único número de 12 bits. Como o expoente vem primeiro na concatenação, ele pesa mais na comparação; a fração só decide o empate.

Isso é **equivalente** a "compare os expoentes; se forem iguais, compare as frações", mas usa **um comparador de 12 bits** em vez de dois comparadores mais lógica de desempate. É menos hardware para o mesmo resultado.

Por que o sinal não entra na comparação? Porque aqui só interessa a magnitude. O sinal é usado no Estágio 3, para decidir entre somar e subtrair, e no final, para determinar o sinal do resultado.

7.2 Estágio 2 — align (alinhamento)

O que faz: iguala os expoentes dos dois números, porque não é possível somar frações que estão em escalas diferentes — do mesmo jeito que não se soma $0,54 \times 10^3$ com $0,87 \times 10^4$ diretamente.

```
exp_diff <= expb - exps;          -- de quantas casas e a diferenca

with exp_diff select
  fracb <= fracs                    when "0000",    -- nao desloca
    "0"      & fracs(7 downto 1)  when "0001",    -- >> 1
    "00"     & fracs(7 downto 2)  when "0010",    -- >> 2
    ...
    "0000000" & fracs(7)         when "0111",    -- >> 7
    "0000000"         when others;    -- >= 8: some
```

Por que subir o expoente do MENOR e não baixar o do maior? Essa é uma pergunta provável. Ao aumentar o expoente do número menor, sua fração precisa ser deslocada para a **direita**, e o que se perde são os bits *menos* significativos — uma perda pequena e aceitável. Se fizéssemos o contrário (baixar o expoente do maior), a fração dele teria que ser deslocada para a **esquerda**, e o bit perdido seria o **mais significativo**, destruindo o número.

O caso when others: quando a diferença de expoentes é 8 ou mais, a fração do menor é zerada por completo. Faz sentido físico: o número menor é tão pequeno em relação ao maior que desaparece na soma. Como a fração tem 8 bits, deslocar 8 ou mais casas à direita não deixa nada mesmo.

7.3 Estágio 3 — add/sub

O que faz: agora que os expoentes são iguais, basta operar as frações.

```
sum <= ('0' & fracb) + ('0' & fracb) when signb = signs else
      ('0' & fracb) - ('0' & fracb);
```

Sinais dos operandos	Operação	Risco
Iguais	soma as frações	pode gerar <i>carry out</i> (o resultado passa de 8 bits)
Diferentes	subtrai as frações	pode gerar zeros à esquerda (o resultado desnormaliza)

Por que os operandos ganham um '0' na frente? Para estender de 8 para 9 bits. A soma de dois números de 8 bits pode chegar a 510, que não cabe em 8 bits. O nono bit ($\text{sum}(8)$) é exatamente o *carry out*, e é ele que o Estágio 4 vai testar primeiro.

Pergunta provável: por que não precisa de complemento de dois?

Porque **a subtração nunca dá resultado negativo** neste circuito. Prova em dois casos:

- **Se $\text{exp_diff} = 0$:** o Estágio 1 já garantiu que $\text{fracb} \geq \text{fracs}$, e sem deslocamento $\text{fracb} = \text{fracs}$. Logo $\text{fracb} \geq \text{fracb}$.
- **Se $\text{exp_diff} \geq 1$:** o deslocamento à direita fez $\text{fracb} \leq 127$ (perdeu o bit 7), enquanto $\text{fracb} \geq 128$ porque está normalizado. Logo $\text{fracb} > \text{fracb}$.

Por isso o circuito não tem lógica de "trocar a ordem dos operandos" nem de complemento de dois. É uma simplificação que **depende inteiramente das entradas estarem normalizadas** — e é exatamente por isso que o pré-normalizador da seção 8.1 é obrigatório nesta versão.

7.4 Estágio 4 — normalize

Este estágio tem uma seção própria (a seção 8), porque é o que a professora mais provavelmente vai querer discutir e é o mais interessante do projeto.

7.5 O sinal do resultado

```
sign_out <= signb;
```

Uma linha só. O sinal do resultado é sempre o sinal do **maior** número. Isso é correto em sinal-magnitude: quem tem maior magnitude "manda" no sinal do resultado. Se os dois têm o mesmo sinal, tanto faz; se têm sinais opostos, o maior vence — que é justamente o comportamento correto da subtração.

7.6 O fluxo completo com números reais (1536 + 320)

Etapa		O que acontece
Registradores		$n^{\circ}1 = 0 \cdot 1011 \cdot 11000000 \cdot n^{\circ}2 = 0 \cdot 1001 \cdot 10100000$
Pré-normalizador		as duas frações já têm MSB = 1 → passa direto , LEDR8 apagado
E1 sort		compara 1011 11000000 com 1001 10100000 → nº1 é o maior . expb=11, fracb=192 · exps=9, frags=160
E2 align		$\text{exp_diff} = 11 - 9 = 2 \rightarrow$ desloca frags duas casas à direita: 160 → 40
E3 add		sinais iguais → soma: $192 + 40 = 232 = 0 \ 1110 \ 1000$. $\text{sum}(8) = 0$, sem carry
E4 normalize		sem carry e sem zeros à esquerda (leado = 0) → não desloca nada. expoente = 11, fração = 232
Display		E8.b → $232 \times 2^3 = 1856$

Onde a precisão se perde: no Estágio 2. Aqui $160 \div 4 = 40$ deu exato, mas se tivesse dado 40,25, o 0,25 seria simplesmente **descartado**. Este projeto **não arredonda** — é uma decisão declarada pelo livro ("*we ignore the round-off error*"). Toda perda de precisão é por **truncamento**, e sempre para baixo em magnitude.


```
leado <= "000" when (sum(7) = '1') else
    "001" when (sum(6) = '1') else
    "010" when (sum(5) = '1') else
    "011" when (sum(4) = '1') else
    "100" when (sum(3) = '1') else
    "101" when (sum(2) = '1') else
    "110" when (sum(1) = '1') else
    "111";
```

Funciona como um **codificador de prioridade**: encontra a posição do 1 mais significativo. Se `sum(7..0)` for todo zero, cai no último `else` e `leado` vale 7.

Pedaço 4.2 — deslocador à esquerda (`sum_norm`)

```
with leado select
    sum_norm <= sum(7 downto 0)          when "000",
    sum(6 downto 0) & "0"                when "001",
    sum(5 downto 0) & "00"              when "010",
    ...
    sum(0) & "0000000" when others;
```

Desloca a fração para a esquerda exatamente a quantidade contada pelo pedaço 4.1, trazendo o 1 mais significativo para a posição do bit 7.

Pedaço 4.3 — a decisão final entre os três casos

```
if sum(8) = '1' then                -- CASO (a): sobrou carry
    expn <= expb + 1;
    fracn <= sum(8 downto 1);       -- equivale a deslocar >> 1

elsif leado > expb then              -- CASO (c): pequeno demais
    expn <= (others => '0');
    fracn <= (others => '0');       -- força zero canônico

else                                -- CASO (b): faltou, desloca esquerda
    expn <= expb - leado;
    fracn <= sum_norm;
end if;
```

Caso	Quando ocorre	O que se faz e por quê
(a) sobrou carry típico após soma	<code>sum(8) = '1'</code>	O resultado passou de 1,xxxx. Desloca a fração uma casa à direita e soma 1 ao expoente. Perdeu metade na fração, ganhou o dobro no expoente — o valor se mantém (menos o bit descartado).
(c) pequeno demais após subtração	<code>leado > expb</code>	Seria preciso deslocar mais casas à esquerda do que o expoente tem de "saldo" para pagar. O valor é menor que a menor magnitude representável, então força zero canônico .
(b) faltou após subtração	demais casos	O resultado tem zeros à esquerda. Desloca à esquerda <code>leado</code> casas e subtrai <code>leado</code> do expoente. Ganhou na fração, pagou no expoente — o valor não muda.

A ideia unificadora, se a professora pedir a intuição: normalizar é sempre uma **troca** entre fração e expoente que *preserva o valor*. Deslocar a fração uma casa à esquerda a multiplica por 2, então é preciso subtrair 1 do expoente para compensar. Deslocar à direita divide por 2, então soma-se 1 ao expoente. O caso (c) é o único em que a troca é impossível, porque o expoente chegou ao mínimo (zero) e não há mais de onde tirar.

8.3 Os três casos, com números

Caso	Conta	O que o Estágio 4 faz	Resultado
(a)	7 + 6,5	<code>sum = 224+208 = 432 = 1 1011 0000</code> , carry. <code>exp = 3+1 = 4</code> , <code>frac = 432>>1 = 216</code>	D8.4 = 13,5 ✓
(b)	9 – 8,5	<code>sum = 144–136 = 8 = 0000 1000</code> , 4 zeros à esquerda. <code>exp = 4–4 = 0</code> , <code>frac = 8<<4 = 128</code>	80.0 = 0,5 ✓
(c)	4,5 – 4,46875	<code>sum = 144–143 = 1</code> , precisaria de 7 deslocamentos mas <code>expb = 3</code> . <code>7 > 3 → força zero</code>	00.0 = 0 ✓

8.4 Três achados sobre a normalização (bom material de relatório)

- Existe "zero negativo".** Em `x + (-x)`, o Estágio 1 desempata pelo `else` e escolhe o operando *negativo* como o maior. O resultado é zero, mas `sign_out = '1'` e o display mostra `-00`. O formato admite duas representações de zero.
- O zero nem sempre sai canônico.** Quando `expb` é grande (por exemplo 8), o cancelamento exato dá `leado = 7`, que *não* é maior que 8. O circuito cai no caso (b) em vez do (c) e devolve `exp = 1`, `frac = 0`. Numericamente é zero ($0/256 \times 2 = 0$), mas não é a forma canônica que o enunciado exige.
- O overflow é silencioso.** No caso (a), se `expb` já vale 15, `expb + 1` estoura os 4 bits e volta para 0. Não existe flag de overflow. Ver seção 9.

9. Os limites do formato

9.1 O teto é 32.640

Este é um ponto que vale ter na ponta da língua, porque é uma limitação estrutural do formato e não do nosso circuito. A culpa é do **expoente de 4 bits**:

```
4 bits -> expoente vai de 0 a 15, e ponto final

maior valor possivel = (255/256) x 2^15 = 255 x 128 = 32.640
menor valor nao nulo = (128/256) x 2^0 = 128 / 256 = 0,5
```

Um número como 60.000 exigiria expoente = 16, e 16 não existe em 4 bits. Não há como representá-lo, e portanto não há como somar até ele.

9.2 O que acontece se tentar mesmo assim

digitado: 32.640 + 32.640
resposta certa: 65.280
display mostra: FF.0 = 0,996

A soma gerou *carry out*, o Estágio 4 mandou fazer expoente + 1, mas 15 + 1 estoura os 4 bits e **volta para 0**. O resultado sai completamente errado, **sem nenhum sinal de alarme**. Não existe flag de overflow neste projeto.

Isso é um bom item de demonstração: mostra que vocês conhecem o limite do que construíram.

9.3 Precisão variável — a tabela de faixas

A fração tem apenas 8 bits, então em cada faixa de expoente existem somente 128 valores possíveis. Quanto maior o número, mais **grosso** fica o passo:

Expoente	Passo (menor diferença)	Faixa representável
15	128	16.384 a 32.640 ← teto
14	64	8.192 a 16.320
13	32	4.096 a 8.160
12	16	2.048 a 4.080
11	8	1.024 a 2.040
10	4	512 a 1.020
9	2	256 a 510
8	1	128 a 255
7	0,5	64 a 127,5
4	0,0625	8 a 15,9375
0	1/256	0,5 a 0,996 ← piso

Consequência prática: perto de 32.000, os números só existem **de 128 em 128**. Por isso 32.512 é representável (254 × 128) mas 20.000 não é (20.000 ÷ 128 = 156,25, não é inteiro).

9.4 Recomendação prática

- Para **números grandes exatos**, escolha múltiplos de 128 entre 16.384 e 32.640, ou múltiplos de 64 entre 8.192 e 16.320.
- Exemplo grande que fecha exato e é bom para demonstração: **16.384 + 16.128 = 32.512** (seção 6.5).
- Para passar de 32.640 a única saída seria **mudar o formato** (por exemplo, expoente de 5 bits, chegando a 2³¹). Mas isso deixa de ser o "ponto flutuante simplificado de 13 bits" que o enunciado define.

10. Perguntas prováveis da professora

Respostas curtas e diretas, para consulta rápida antes ou durante a apresentação.

P1. Como vocês fizeram a soma?

Em quatro blocos combinacionais encadeados, seguindo o mesmo procedimento de somar à mão em notação científica: **ordenar** (achar quem tem maior magnitude), **alinhar** (igualar os expoentes deslocando a fração do menor à direita), **somar ou subtrair** as frações (dependendo dos sinais), e **normalizar** o resultado. Detalhes na seção 7.

P2. Como vocês fizeram a normalização?

Em dois lugares. Na **saída**, o Estágio 4 do livro, com três pedaços de lógica: contador de zeros à esquerda (codificador de prioridade), deslocador à esquerda, e uma decisão entre três casos — carry out, zeros à esquerda, e pequeno demais para normalizar. Na **entrada**, nós acrescentamos um **pré-normalizador**, porque a digitação livre eliminou a garantia que as constantes do livro davam. Detalhes na seção 8.

P3. Por que o somador não tem clock?

Porque a operação é puramente combinacional: dados os dois operandos, o resultado é uma função lógica direta deles. O único elemento sequencial do projeto são os quatro registradores que guardam os operandos digitados — e eles existem por limitação de *pinos*, não da matemática.

P4. Por que a comparação do Estágio 1 usa `exp & frac` concatenados?

Porque com o expoente na frente da concatenação, um único comparador de 12 bits faz o trabalho de "compare os expoentes; se empatarem, compare as frações". Economiza um comparador e toda a lógica de desempate.

P5. Por que a subtração não precisa de complemento de dois?

Porque ela nunca dá negativo. Se os expoentes são iguais, o Estágio 1 garantiu `fracb ≥ fracf`. Se são diferentes, o alinhamento deixou `fracf ≤ 127` enquanto `fracb ≥ 128` por estar normalizado. A prova completa está na seção 7.3 — e ela é justamente o motivo de o pré-normalizador ser obrigatório.

P6. Por que vocês acrescentaram um bloco que não está no livro?

Porque no livro a fração dos operandos era montada com o bit mais significativo cravado em '1' no código, o que garantia entrada normalizada. Nossa versão permite digitar os 26 bits livremente, então essa garantia sumiu — e sem ela o somador produz lixo silenciosamente. Temos o caso medido: um operando não normalizado fazia o display mostrar `+386` em vez de `-126,5`.

P7. Por que o `disp_mux` foi removido?

Na placa do livro, os quatro displays compartilhavam as linhas de segmento, então era obrigatório multiplexar no tempo — e isso exigia um clock de 50 MHz só para varrer os dígitos. A DE10-Lite tem **seis displays com pinos próprios**, cada um com seus oito sinais. Cada display recebe seu próprio decodificador e pronto.

P8. Por que o resultado às vezes perde 1?

Porque o projeto **não arredonda**: bits deslocados para fora são descartados. É uma decisão declarada pelo livro. Exemplo: `149 + 128 = 277`, mas o circuito devolve `276` — o bit descartado no deslocamento do Estágio 4.

P9. O que acontece se somar os dois maiores números?

Overflow silencioso. O carry manda fazer `expoente + 1` com o expoente já em 15; os 4 bits estouram e o expoente volta a 0. O resultado sai errado sem nenhum aviso — não existe flag de overflow. Podemos demonstrar na placa.

P10. Existe mais de uma representação para o zero?

Sim, duas situações. Primeiro, **zero negativo**: em `x + (-x)` o Estágio 1 desempata pelo `else` e adota o sinal do operando negativo, então o display mostra `-00`. Segundo, **zero não canônico**: com expoente de partida alto, o cancelamento exato devolve `exp = 1, frac = 0`, que vale zero mas não é a forma canônica exigida.

P11. Quantos bits de entrada o somador precisa, e quantos a placa oferece?

O somador precisa de **26 bits** (13 por operando). A DE10-Lite oferece **12 entradas manuais** (10 chaves + 2 botões). Por isso os operandos são digitados em quatro etapas e guardados em registradores — as mesmas oito chaves são reaproveitadas quatro vezes.

P12. Como funciona o anti-repique do botão?

Um contador divide o clock de 50 MHz e o botão é observado apenas uma vez a cada **10 ms**. Os saltos mecânicos do contato (tipicamente abaixo de 5 ms) acontecem entre duas amostragens e simplesmente não são vistos. Sobre isso há um detector de borda, então um clique produz exatamente uma carga, mesmo se o botão ficar segurado. Há também um sincronizador de dois flip-flops, porque os botões são assíncronos em relação ao clock.

P13. Como vocês validaram o projeto?

Com dois testbenches auto-verificáveis, que imprimem PASS/FAIL por caso e param a simulação com `severity failure` se algo

falhar. Um valida o pré-normalizador (7 casos), o outro valida a máquina de carga completa com clock, as quatro fases e os displays (9 casos). Os valores esperados foram conferidos independentemente num modelo em Python. Detalhes na seção 12.

P14. Vocês alteraram a lógica matemática do livro?

Não. Os quatro estágios do `fp_adder` são idênticos ao Listing 3.19. A única mudança é cosmética: a comparação do Estágio 1 usa `unsigned(...)` explícito em vez de comparar `std_logic_vector` direto — o hardware sintetizado é o mesmo. Todas as adaptações estão fora do núcleo: no top-level, nos registradores e no pré-normalizador.

11. O que foi mudado em relação ao original

11.1 Removemos

- O módulo `disp_mux` e todo o esquema de multiplexação no tempo dos displays.
- A saída `an` (anodos), que não existe na DE10-Lite.
- As declarações de `component` — usamos instânciação direta de entidade (`entity work.xxx`), recurso do VHDL-93 que dispensa a duplicação de interface.
- O circuito de teste com operandos constantes do Listing 3.20, substituído pela entrada sequencial completa.

11.2 Acrescentamos

- **Quatro registradores** para guardar os dois operandos completos (26 bits).
- **Máquina de carga:** sincronizador de dois estágios, divisor de clock para anti-ripple de 10 ms, e detector de borda.
- **Pré-normalizador** (dois deles, um por operando) — o bloco mais importante da nossa adaptação, ver seção 8.1.
- **Modo conferência** no KEY1, para verificar os operandos guardados antes de concluir a conta.
- **Indicador de entrada ajustada** no LEDR8.
- Restrição de tempo (`.sdc`), necessária porque agora existe clock.

11.3 Adaptamos

Item	Original (placa do livro)	Nossa versão (DE10-Lite)
Chaves disponíveis	8 (<code>sw7</code> . . <code>sw0</code>)	10 (<code>SW9</code> . . <code>SW0</code>)
Botões disponíveis	4 (<code>btn3</code> . . <code>btn0</code>)	2 (<code>KEY1</code> , <code>KEY0</code>)
Polaridade dos botões	ativo em nível alto	ativo em nível baixo — invertido no VHDL
Displays 7 segmentos	4, com segmentos compartilhados	6, cada um com pinos próprios
Largura do display	7 bits	8 bits (7 segmentos + ponto decimal)
Entrada dos operandos	constantes no código + chaves diretas	digitação completa em 4 fases, com registradores
Clock	necessário para o <code>disp_mux</code>	necessário para a carga dos registradores

11.4 Não mudamos

Nada da lógica matemática do `fp_adder`. Os quatro estágios são idênticos ao Listing 3.19 do livro. Isso é proposital: o objetivo do trabalho é *validar* o projeto teórico e adaptá-lo ao hardware novo, não reescrevê-lo. Todas as adaptações ficaram fora do núcleo.

12. Validação: como sabemos que funciona

12.1 Dois testbenches auto-verificáveis

Ambos imprimem PASS/FAIL por caso, imprimem um RESUMO no final, e param a simulação com `severity failure` se qualquer caso falhar. Portanto "rodou até o fim sem erro" já é, por si só, evidência de validação.

Testbench	O que valida	Casos
tb_pre_normalizador	Que a saída sempre tem <code>frac(7)='1'</code> ou é o zero canônico, e que o valor numérico não muda — exceto quando é genuinamente pequeno demais.	7
tb_somador_pf_de10lite_seq	A máquina de carga completa: clock, anti-ripique, detector de borda, as quatro fases, o somador e os seis displays. É o teste que pega fase trocada, carga dupla e nibble no display errado.	9

12.2 O que cada caso cobre

Caso	Por que ele existe
Já normalizado, passa direto	garante que o pré-normalizador não estraga entrada válida
2 e 7 zeros à esquerda	o deslocamento e o pagamento no expoente, preservando o valor
Fração zero com expoente $\neq 0$	conversão para o zero canônico
Subnormal (desloc > expoente)	a regra 3, que evita o resultado impossível
Limite: desloc = expoente	caso de fronteira entre as regras 3 e 4
Estado inicial (128 + 128)	os valores de fábrica dos registradores
Carry out (7 + 6,5)	Estágio 4, caso (a) — exemplo 4 do livro
Deslocamento à esquerda (9 – 8,5)	Estágio 4, caso (b) — exemplo 2 do livro
Pequeno demais → zero	Estágio 4, caso (c) — era impossível de alcançar na versão com constantes ; só ficou testável com a entrada livre
Entrada não normalizada	prova que o pré-normalizador conserta: o caso que dava +386 agora dá -126,5 exato
Segurar o KEY0 por 200 ciclos	prova que um clique = uma carga
Conferência dos dois operandos	o multiplexador de exibição e o KEY1

12.3 Verificação independente

Todos os valores esperados dos testbenches foram calculados por um **modelo independente em Python** que reimplementa os quatro estágios e o pré-normalizador bit a bit. Os 16 casos batem. Isso protege contra o erro clássico de escrever no testbench o valor que o circuito produz em vez do valor que ele *deveria* produzir.

12.4 Comandos de simulação

GHDL + GTKWave

```
ghdl -a --std=08 somador_pf_de10lite_seq.vhd
ghdl -a --std=08 tb_somador_pf_de10lite_seq.vhd

ghdl -e --std=08 tb_pre_normalizador
ghdl -r --std=08 tb_pre_normalizador

ghdl -e --std=08 tb_somador_pf_de10lite_seq
ghdl -r --std=08 tb_somador_pf_de10lite_seq --wave=onda_seq.ghw

gtkwave onda_seq.ghw
```

Questa

```
vlib work
vmap work work
vcom -2008 somador_pf_de10lite_seq.vhd
vcom -2008 tb_somador_pf_de10lite_seq.vhd
vsim work.tb_pre_normalizador
run -all
vsim work.tb_somador_pf_de10lite_seq
add wave -r /*
run -all
```

Para a captura do Estágio 4 (pedida na Etapa 1 do enunciado), coloque no visualizador de ondas os sinais internos do `fp_adder`: `sum`, `leado`, `sum_norm`, `expb`, `expn` e `fracn`. Os casos de carry out, deslocamento à esquerda e pequeno-demaís aparecem em sequência e mostram os três caminhos da decisão 4.3.

13. Problemas comuns

Sintoma	Causa provável	O que fazer
Displays todos apagados	não gravou, ou placa sem energia	confira o LED POWER e regrave o .sof
Todos os segmentos acesos em HEX4 e HEX5	pinos não dirigidos	confirme que importou o .qsf completo , com HEX3, HEX4 e HEX5
USB-Blaster não aparece no Hardware Setup	driver (Windows) ou regra udev (Linux)	Windows: aponte o driver em quartus/drivers/usb-blaster. Linux: instale o 51-usbbaster.rules e reconecte o cabo
Projeto compila mas nada funciona na placa	clock no pino errado	verifique no Pin Planner que MAX10_CLK1_50 está em PIN_P11
Um clique carrega duas vezes	não deve acontecer (anti-repique de 10 ms)	se acontecer, aumente DIV_DEBOUNCE de 500000 para 1000000 e recompile
Resultado estranho e LEDR8 aceso	fração digitada com SW7 para baixo	não é bug: o pré-normalizador reescreveu seu número. Deixe SW7 sempre para cima
Resultado estranho e LEDR8 apagado	faltou carregar alguma das 4 fases	segure KEY1 e confira os dois operandos com SW9 baixo e SW9 cima
Número grande dá resultado minúsculo	overflow do expoente	o resultado passou de 32.640. Não há solução dentro deste formato
Display não muda ao virar as chaves	é o comportamento esperado	as chaves só entram no circuito quando o KEY0 é apertado
HEX5 não mostra a fase certa	SW9/SW8 na posição errada	1 = baixo baixo · 2 = baixo cima · 3 = cima baixo · 4 = cima cima
Resultado muda "sozinho" durante a digitação	é o comportamento esperado	tudo depois dos registradores é combinacional; só confie após os 4 cliques

14. Arquivos do projeto

Arquivo	O que é
somador_pf_de10lite_seq.vhd	O programa. Autocontido — é o único arquivo de código que vai para o Quartus. Contém quatro blocos: <code>hex_to_sseg</code> (decodificador de 7 segmentos), <code>pre_normalizador</code> (garante entrada válida), <code>fp_adder</code> (o somador de 4 estágios) e <code>somador_pf_de10lite_seq</code> (o top-level). Cada bloco tem um cabeçalho de comentário explicando o que faz e por quê.
tb_somador_pf_de10lite_seq.vhd	Os dois testbenches auto-verificáveis. Não vai para o Quartus, apenas para a simulação.
somador_pf_de10lite_seq.qsf	Atribuição de pinos completa: clock, 10 chaves, 2 botões, 10 LEDs e os 6 displays com seus pontos decimais.
somador_pf_de10lite_seq.sdc	Restrição de tempo. Obrigatório porque esta versão tem clock.
MANUAL_DE_USO.md	Versão em Markdown deste documento, para o repositório.

14.1 Estrutura interna do programa

```
VOCE posiciona SW7..SW0 e escolhe o campo com SW9/SW8
      |
      v
+-----+
| KEY0 -> sincronizador -> divisor 10ms -> | <-- unica logica
|      detector de borda                    | sincrona
+-----+
      v
+-----+
| 4 REGISTRADORES                          |
| no1: sinal / expoente / fracao           |
| no2: sinal / expoente / fracao           |
+-----+
      v
+-----+
| PRE-NORMALIZADOR (um para cada numero)   |
| garante fracao com MSB=1, sem mudar o valor |
+-----+
      v
===== AQUI COMECA O SOMADOR =====
      v
ESTAGIO 1 sort -> ESTAGIO 2 align ->
ESTAGIO 3 add/sub -> ESTAGIO 4 normalize
      v
+-----+
| MULTIPLEXADOR DE EXIBICAO                <-- KEY1 |
| solto = resultado | apertado = operando         |
+-----+
      v
      HEX0..HEX5 e LEDR
```

15. Resumo de bolso

Converter DECIMAL para as CHAVES

1. Separe o sinal (negativo -> SW4 para cima).
2. Divida por 2 (se ≥ 256) ou multiplique por 2 (se < 128)
ate cair entre 128 e 255.
3. Esse numero final e a FRACAO.
4. EXPOENTE = 8 + divisoes (ou 8 - multiplicacoes)
5. Converta os dois para binario.
6. Expoente fora de 0..15 -> o numero NAO CABE.

Digitar: 4 cliques no KEY0

SW9 baixo, SW8 baixo -> sinal(SW4) + expoente(SW3..SW0) do no 1
SW9 baixo, SW8 cima -> fracao(SW7..SW0) do no 1
SW9 cima, SW8 baixo -> sinal + expoente do no 2
SW9 cima, SW8 cima -> fracao do no 2

Na fase de fracao: SW7=128 SW6=64 SW5=32 SW4=16
SW3=8 SW2=4 SW1=2 SW0=1
SW7 SEMPRE PARA CIMA

Na fase de sinal+exp: SW4=sinal (cima = negativo)
SW3=8 SW2=4 SW1=2 SW0=1
SW7, SW6, SW5 ignoradas

Ler o DISPLAY em decimal

FRACAO = (digito do HEX2) x 16 + (digito do HEX1)

expoente > 8 -> multiplique a fracao por 2, (exp - 8) vezes
expoente < 8 -> divida a fracao por 2, (8 - exp) vezes
expoente = 8 -> o valor E a fracao

HEX3 com traco = negativo
LEDR7..0 = a mesma fracao em binario (128 64 32 16 8 4 2 1)

Números importantes

TETO = 32.640 (0.11111111×2^{15})
PISO = 0,5 (0.10000000×2^0)
Fracao sempre entre 128 e 255 quando normalizada.
Expoente 8 -> o valor E a fracao.

Botões

KEY0 = carrega o campo selecionado (1 clique = 1 carga)
KEY1 = segurar para CONFERIR o operando escolhido por SW9
(aparece um "C" no HEX4)

Se algo der errado na apresentação

LEDR8 aceso -> voce digitou fracao com SW7 para baixo
resultado esquisito -> segure KEY1 e confira os dois operandos
numero grande deu
resultado minusculo -> passou de 32.640, estourou o expoente
nada muda nas
chaves -> normal, so muda quando aperta KEY0